

JavaScript Event Loop

A QUICK REFERENCE

The Most Complete Full Stack
Developer Roadmap

www.thedevspace.io

WHAT ARE MACROTASKS

Macrotasks (or just "tasks") are scheduled work items dispatched from the task queue.

Each macrotask runs to completion and yields control back to the event loop.



```
1 setTimeout(() => console.log("This is a macrotask"), 1000);
```

Each macrotask gets a microtask checkpoint after it finishes.

WHAT ARE MICROTASKS

Microtasks (also called "jobs") run between macrotasks.

They are small follow-up work and always run after the current macrotask, and before the next macrotask.



```
1 Promise.resolve()  
2   .then(() => console.log("This is a microtask."))  
3   .then(() => console.log("This is another microtask."));
```

Microtasks are processed FIFO until the microtask queue is empty. Long microtask can delay subsequent macrotasks.

THE EVENT LOOP

1. Run the current macrotask (e.g., initial script, event handler, timer callback)
2. Process all microtasks in FIFO order
3. Optionally perform rendering (browser repaint)
4. Process the next macrotask
5. Repeat

Consider this example





```
1 console.log("sync");
2
3 setTimeout(() => console.log("macrotask1"), 0);
4
5 Promise.resolve()
6   .then(() => console.log("micro1"))
7   .then(() => console.log("micro2"));
8
9 setTimeout(() => console.log("macrotask2"), 1000);
10
11 Promise.resolve()
12   .then(() => console.log("micro3"))
13   .then(() => console.log("micro4"));
14
15 setTimeout(() => console.log("macrotask3"), 1000);
```

**What do you think is
the output?**

**Async task runs first**

```
1 console.log("sync");  
2  
3 setTimeout(() => console.log("macrotask1"), 0);  
4  
5 Promise.resolve()  
6   .then(() => console.log("micro1"))  
7   .then(() => console.log("micro2"));  
8  
9 setTimeout(() => console.log("macrotask2"), 1000);  
10  
11 Promise.resolve()  
12   .then(() => console.log("micro3"))  
13   .then(() => console.log("micro4"));  
14  
15 setTimeout(() => console.log("macrotask3"), 1000);
```



Macro tasks run later

```
1 console.log("sync");
2
3 setTimeout(() => console.log("macrotask1"), 0);
4
5 Promise.resolve()
6   .then(() => console.log("micro1"))
7   .then(() => console.log("micro2"));
8
9 setTimeout(() => console.log("macrotask2"), 1000);
10
11 Promise.resolve()
12   .then(() => console.log("micro3"))
13   .then(() => console.log("micro4"));
14
15 setTimeout(() => console.log("macrotask3"), 1000);
```





Queue first micro task

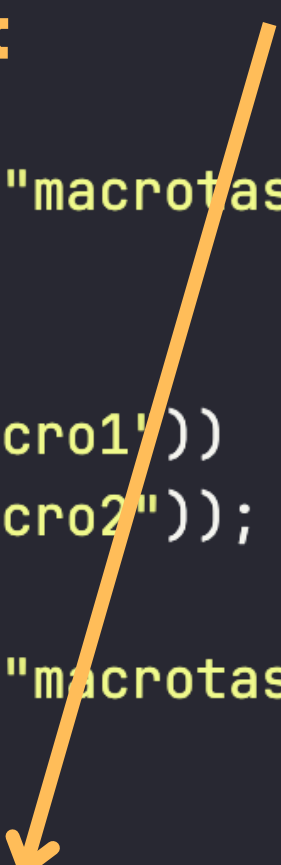
```
1 console.log("sync");
2
3 setTimeout(() => console.log("macrotask1"), 0);
4
5 Promise.resolve()
6   .then(() => console.log("micro1"))
7   .then(() => console.log("micro2"));
8
9 setTimeout(() => console.log("macrotask2"), 1000);
10
11 Promise.resolve()
12   .then(() => console.log("micro3"))
13   .then(() => console.log("micro4"));
14
15 setTimeout(() => console.log("macrotask3"), 1000);
```





Queue second micro task

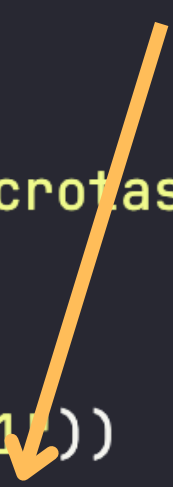
```
1 console.log("sync");
2
3 setTimeout(() => console.log("macrotask1"), 0);
4
5 Promise.resolve()
6   .then(() => console.log("micro1"))
7   .then(() => console.log("micro2"));
8
9 setTimeout(() => console.log("macrotask2"), 1000);
10
11 Promise.resolve()
12   .then(() => console.log("micro3"))
13   .then(() => console.log("micro4"));
14
15 setTimeout(() => console.log("macrotask3"), 1000);
```





Queue third micro task

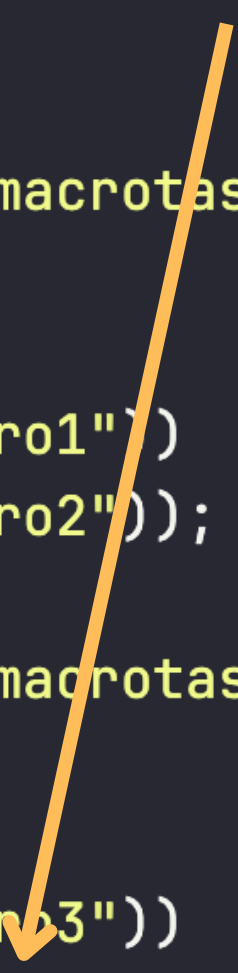
```
1 console.log("sync");
2
3 setTimeout(() => console.log("macrotask1"), 0);
4
5 Promise.resolve()
6   .then(() => console.log("micro1"))
7   .then(() => console.log("micro2"));
8
9 setTimeout(() => console.log("macrotask2"), 1000);
10
11 Promise.resolve()
12   .then(() => console.log("micro3"))
13   .then(() => console.log("micro4"));
14
15 setTimeout(() => console.log("macrotask3"), 1000);
```





Queue last micro task

```
1 console.log("sync");
2
3 setTimeout(() => console.log("macrotask1"), 0);
4
5 Promise.resolve()
6   .then(() => console.log("micro1"))
7   .then(() => console.log("micro2"));
8
9 setTimeout(() => console.log("macrotask2"), 1000);
10
11 Promise.resolve()
12   .then(() => console.log("micro3"))
13   .then(() => console.log("micro4"));
14
15 setTimeout(() => console.log("macrotask3"), 1000);
```



So the output is...

sync

micro1

micro3

micro2

micro4

macrotask1

macrotask2

macrotask3